



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

SQL Fundamental for Beginners

TGS Ref No: TGS-2020505790

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 12

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
11.0	16 Oct 2020	Legacy learner materials accompanying the v11 slide deck.	Dr. Alfred Ang
12	8 August 2026	Full rebuild in the single-source pipeline: detailed step-by-step for all 11 labs (the slide deck is visual-only), SQL quick reference, glossary, PP + OQ assessment alignment.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
How You Will Be Assessed.....	4
Before You Start — Environment Setup.....	4
Topic 01 — Data Modeling.....	5
Lab 1 — Set Up SQLite Studio & Import the World Database.....	5
Lab 2 — Create a Database and Tables.....	7
Lab 3 — Model Data with Constraints and Keys.....	9
Topic 02 — Data Processing and Analysis.....	11
Lab 4 — Query Data with SELECT.....	12
Lab 5 — Filter with SQL Operators.....	13
Lab 6 — Insert, Update and Delete Records.....	15
Topic 03 — Data Transformation.....	17
Lab 7 — Aggregate Data with COUNT, AVG and SUM.....	17
Lab 8 — Join Data from Multiple Tables.....	19
Lab 9 — Group Data with GROUP BY and HAVING.....	21
Topic 04 — Introduction to Data Warehouse.....	23
Lab 10 — Map an E-R Model to Database Tables (SMRT Case Study).....	23
Lab 11 — Author Stored Procedures for SQL Server.....	26
SQL Quick Reference.....	28
After the Course.....	29
Glossary.....	30

Introduction

This Learner Guide accompanies the WSQ course SQL Fundamental for Beginners (TGS-2020505790), conducted by Tertiary Infotech Academy Pte Ltd. It provides detailed step-by-step instructions for all 11 hands-on labs, organised by the four course topics: Data Modeling, Data Processing and Analysis, Data Transformation, and Introduction to Data Warehouse. Every lab runs in SQLite Studio (or the free sqliteonline.com) against the world sample database.

Use this guide alongside the course slides and the lab files in the `courseware/labs/` folder of the course repository. The slides present each concept and lab visually; THIS guide carries the full step-by-step SQL for you to type and run. The guide is an approved open-book reference during the final assessment.

Course Learning Outcomes

- LO1: Apply data modeling for business processes.
- LO2: Apply data processing and analysis using SQL.
- LO3: Apply data transformation from multiple data sources.
- LO4: Apply data mapping to data warehouse.

Skills Framework: Data Engineering (ICT-DIT-3005-1.1), ICT Skills Framework.

How You Will Be Assessed

The final assessment is conducted at the end of the training day and consists of two instruments:

- Practical Performance (PP) — hands-on SQL tasks, 70 minutes, open book, assessor-to-candidate ratio 1:10.
- Oral Questioning (OQ) — 20 minutes, one-to-one with the assessor.
- Format: Open Book — the course slides, this Learner Guide and other approved materials may be used.
- A minimum of 75% attendance is required and learners must be assessed as 'Competent' to be eligible for funding.
- You must take the Assessment digital attendance (TRAQOM) before the assessment begins, and sign the Assessment Summary Record after the Oral Questioning.
- An appeal process is available if required — speak to the trainer or the administrator.

Before You Start — Environment Setup

What you need

- SQLite Studio — free, open-source, cross-platform. Download from <https://sqlitestudio.pl> and install for your OS.

- No-install alternative: the free cloud editor at <https://sqliteonline.com> (also offers an MS SQL engine for the stored-procedure lab).
- The world sample database (world.db) and the insert-data script (world-inserts.sql) — download both from the course materials on <https://lms-tms.tertiaryinfotech.com>.
- Any laptop OS — Windows, macOS or Linux.

Set up SQLite Studio

Install SQLite Studio, then connect a database: Database → Add a database, browse to a .db file (or click the green '+' to create a new one) and Connect to the database from the left-hand Databases panel. Open the query editor with Tools → Open SQL editor and check the active database shown at the top of the editor.

Conventions used in every lab

- SQL statements are shown in code blocks — type or paste them into the SQL editor and press the Execute (▶ / F9) button.
- SQL keywords are shown in UPPERCASE by convention; SQLite accepts any case.
- Statements end with a semicolon; you can run several statements together.
- Labs 1–10 use SQLite; Lab 11 (stored procedures) uses SQL Server syntax — author it on sqliteonline.com's MS SQL engine.
- If a query fails, read the error message at the bottom of the editor — it names the offending token.

Topic 01 — Data Modeling

Data Sources · Data Modeling · SQL Data Types · Create & Manage Databases (TSC mapping: A1, A2, K1, K2)

Key concepts

- Relational databases — A database is one or more tables; each table is rows (records) and columns (fields) with a fixed row type.
- DDL statements — CREATE / ALTER / DROP define databases, tables, indexes and their columns and data types.
- Constraints — NOT NULL, DEFAULT, UNIQUE, CHECK, PRIMARY KEY and FOREIGN KEY enforce rules on the data.
- Data modeling — Primary and foreign keys link tables into entity relationships that mirror the business process.

Lab 1 — Set Up SQLite Studio & Import the World Database

Objective: LO1 — identify relevant data sources and set up the SQL workbench (A1, A2).

Goal: Install SQLite Studio, connect it to the world sample database and explore its tables — your data source for the whole course.

What you'll build

A working SQLite Studio with the world database connected, showing the City, Country and CountryLanguage tables. (Tools: SQLite Studio, world.db sample database, LMS course materials.)

The screenshot shows the SQLite Studio interface. At the top, the title bar reads 'SQLite Studio — SQL Editor' and 'Database: world.db'. Below the title bar is a menu bar with 'Execute (F9)', 'Database', 'Tools', and 'View'. The main area is divided into two sections: 'QUERY' and 'RESULTS'. The 'QUERY' section contains the following SQL code:

```
QUERY
SELECT * FROM City
LIMIT 5;
```

The 'RESULTS' section displays a table with 5 rows and 5 columns: ID, Name, CountryCode, District, and Population. The data is as follows:

ID	Name	CountryCode	District	Population
1	Kabul	AFG	Kabul	1780000
2	Qandahar	AFG	Qandahar	237500
3	Herat	AFG	Herat	186800
4	Mazar-e-Sharif	AFG	Balkh	127800
5	Amsterdam	NLD	Noord-Holland	731200

At the bottom of the interface, a status bar shows a green dot and the text 'Query executed successfully · 5 rows returned · world database connected'. On the right side of the status bar, it says 'SQL Fundamental for Beginners · TGS-2020505790'.

Figure 1 — Lab 1: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Download SQLite Studio for your OS from the official site and install it. (Alternative: use the free cloud version at sqliteonline.com — no install needed.)

<https://sqlitestudio.pl>

Step 2. Download the world sample database (world.db) from the course materials on the LMS portal.

<https://lms-tms.tertiaryinfotech.com>

Step 3. Open SQLite Studio, click Database → Add a database, browse to the downloaded world.db and click OK.

Step 4. In the left-hand Databases panel, right-click world and choose Connect to the database.

Step 5. Expand the database tree: you should see the City, Country and CountryLanguage tables. Double-click City and open the Data tab to preview its rows.

Test it

The Databases panel shows the connected world database with the City, Country and CountryLanguage tables, and the Data tab displays city rows (ID, Name, CountryCode, District, Population).

Note: The same steps are in courseware/labs/lab-01-*.md in the course repository.

Lab 2 — Create a Database and Tables

Objective: LO1 — create and manage databases and tables with SQL DDL (A1, A2).

Goal: Create your own database named 'sample' and build the City, Country and CountryLanguage tables with CREATE TABLE, choosing an appropriate data type for every column.

What you'll build

A new 'sample' database containing three empty tables whose columns, data types and defaults mirror the world schema. (Tools: SQLite Studio, SQL Editor.)

The screenshot shows the SQLite Studio SQL Editor interface. The title bar reads 'SQLite Studio — SQL Editor' and 'Database: sample.db'. The menu bar includes 'Execute (F9)', 'Database', 'Tools', and 'View'. Below the menu bar, the text 'Lab 2 — Create a Database and Tables' is displayed. The main editor area contains the following SQL query:

```
QUERY
DROP TABLE IF EXISTS City;
CREATE TABLE City (
  ID INTEGER PRIMARY KEY,
  Name TEXT NOT NULL DEFAULT '',
  CountryCode TEXT NOT NULL DEFAULT '',
  District TEXT NOT NULL DEFAULT '',
  Population INTEGER NOT NULL DEFAULT '0'
);
```

Below the query editor, the 'RESULTS' section shows a table with the following data:

name	type	notnull	dfit_value	pk
ID	INTEGER	0	NULL	1
Name	TEXT	1	"	0
CountryCode	TEXT	1	"	0
District	TEXT	1	"	0
Population	INTEGER	1	'0'	0

At the bottom of the interface, a status bar shows a green dot and the text 'Table City created · schema shown from the Structure tab'. On the right side of the status bar, it says 'SQL Fundamental for Beginners · TGS-2020505790'.

Figure 2 — Lab 2: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. In SQLite Studio click Database → Add a database, then click the green '+' (plus) icon, name the new database file sample and save it. Connect to it from the Databases panel.

Step 2. Open the SQL editor: go to Tools → Open SQL editor. Make sure the active database (top of the editor) is sample.

Step 3. Create the City table — run the statement, noting the INTEGER primary key and NOT NULL + DEFAULT constraints.

```
DROP TABLE IF EXISTS City;

CREATE TABLE City (
    ID INTEGER PRIMARY KEY,
    Name TEXT NOT NULL DEFAULT '',
    CountryCode TEXT NOT NULL DEFAULT '',
    District TEXT NOT NULL DEFAULT '',
    Population INTEGER NOT NULL DEFAULT '0'
);
```

Step 4. Create the Country table with columns for code, name, continent, region, surface area, population and government details.

```
DROP TABLE IF EXISTS Country;

CREATE TABLE Country (
    Code TEXT NOT NULL DEFAULT '',
    Name TEXT NOT NULL DEFAULT '',
    Continent TEXT NOT NULL DEFAULT 'Asia',
    Region TEXT NOT NULL DEFAULT '',
    SurfaceArea REAL DEFAULT NULL,
    IndepYear INTEGER DEFAULT NULL,
    Population INTEGER DEFAULT NULL,
    LifeExpectancy REAL DEFAULT NULL,
    GNP REAL DEFAULT NULL,
    LocalName TEXT NOT NULL DEFAULT '',
    GovernmentForm TEXT NOT NULL DEFAULT '',
    Capital INTEGER DEFAULT NULL,
    Code2 TEXT NOT NULL DEFAULT ''
```



```
PRIMARY KEY (Code)
);
```

Step 5. Create the CountryLanguage table — note the composite primary key on (CountryCode, Language).

```
DROP TABLE IF EXISTS CountryLanguage;

CREATE TABLE CountryLanguage (
    CountryCode TEXT NOT NULL DEFAULT '',
    Language TEXT NOT NULL DEFAULT '',
    IsOfficial INTEGER NOT NULL DEFAULT '0',
    Percentage NUMERIC NOT NULL DEFAULT '0.0',
    PRIMARY KEY (CountryCode, Language)
);
```

Step 6. Right-click the sample database and choose Refresh — the three new tables appear. Open each table's Structure tab to verify the columns and data types.

Test it

The sample database lists City, Country and CountryLanguage; each table's Structure tab shows the declared columns, data types, defaults and primary keys.

Note: The same steps are in courseware/labs/lab-02-*.md in the course repository.

Lab 3 — Model Data with Constraints and Keys

Objective: LO1 — apply data modeling for business processes with constraints, keys and relationships (A1, A2).

Goal: Build a Persons–Orders mini-model: enforce data rules with NOT NULL, UNIQUE, DEFAULT and CHECK, link the tables with a PRIMARY KEY / FOREIGN KEY relationship, and speed up searches with an index.

What you'll build

A two-table data model (Persons ← Orders) with working constraints, a foreign-key relationship and an index. (Tools: SQLite Studio, SQL Editor.)

SQLite Studio — SQL Editor

Database: sample.db

Execute (F9)

Database

Tools

View

Lab 3 — Model Data with Constraints and Keys

QUERY

```

CREATE TABLE Persons (
  ID INTEGER NOT NULL UNIQUE,
  LastName TEXT NOT NULL,
  FirstName TEXT,
  Age INTEGER CHECK (Age >= 18),
  City TEXT DEFAULT 'Singapore',
  PRIMARY KEY (ID)
);

INSERT INTO Persons (ID, LastName, FirstName, Age)
VALUES 1, 'Tan', 'Alice', 30

```

RESULTS

ID	LastName	FirstName	Age	City
1	Tan	Alice	30	Singapore

1 row inserted · City filled from DEFAULT · CHECK and UNIQUE constraints active

SQL Fundamental for Beginners · TGS-2020505790

Figure 3 — Lab 3: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. In the sample database's SQL editor, create a Persons table that enforces NOT NULL, UNIQUE, DEFAULT and CHECK rules.

```

CREATE TABLE Persons (
  ID INTEGER NOT NULL UNIQUE,
  LastName TEXT NOT NULL,
  FirstName TEXT,
  Age INTEGER CHECK (Age >= 18),
  City TEXT DEFAULT 'Singapore',
  PRIMARY KEY (ID)
);

```

Step 2. Prove the constraints work: this insert succeeds and fills City with the default value.

```

INSERT INTO Persons (ID, LastName, FirstName, Age)
VALUES (1, 'Tan', 'Alice', 30);

SELECT * FROM Persons;

```

Step 3. Now try to break the rules — each of these statements must FAIL (duplicate ID, NULL LastName, under-age CHECK).

```
INSERT INTO Persons (ID, LastName, Age) VALUES (1, 'Lim', 25);
```

```
INSERT INTO Persons (ID, Age) VALUES (2, 40);
```

```
INSERT INTO Persons (ID, LastName, Age) VALUES (3, 'Lee', 15);
```

Step 4. Create the Orders table whose PersonID column is a FOREIGN KEY referencing Persons — the child table pointing at the parent.

```
CREATE TABLE Orders (  
    OrderID INTEGER NOT NULL,  
    OrderNumber INTEGER NOT NULL,  
    PersonID INTEGER,  
    PRIMARY KEY (OrderID),  
    FOREIGN KEY (PersonID) REFERENCES Persons(ID)  
);
```

Step 5. Add a column to an existing table with ALTER TABLE.

```
ALTER TABLE Persons ADD Email TEXT;
```

Step 6. Create an index so searches on LastName are fast.

```
CREATE INDEX idx_lastname ON Persons (LastName);
```

Test it

The valid insert appears in Persons with City = 'Singapore'; the three rule-breaking inserts each raise a constraint error; Orders shows a foreign key to Persons in its DDL; and idx_lastname is listed under Indexes.

Note: The same steps are in courseware/labs/lab-03-*.md in the course repository.

Topic 02 — Data Processing and Analysis

SQL Queries · SQL Operators · Insert, Update & Delete (TSC mapping: A3, A5, K4)

Key concepts

- Queries — SELECT retrieves data into a result-set; DISTINCT removes duplicates; WHERE filters rows.

- Operators — AND/OR/NOT, IN, BETWEEN, LIKE wildcards and IS NULL build precise filter conditions.
- Shaping results — ORDER BY sorts, LIMIT caps the row count, and aliases (AS) rename columns and tables.
- Changing data — INSERT INTO adds records, UPDATE modifies them and DELETE removes them — always with WHERE.

Lab 4 — Query Data with SELECT

Objective: LO2 — apply data processing and analysis using SQL queries (A3, A5).

Goal: Query the world database with SELECT: retrieve all columns, chosen columns, distinct values and filtered rows, then combine sorting and limits to answer real questions.

What you'll build

A set of working queries answering questions about world cities and countries, run against the world database. (Tools: SQLite Studio, world database.)

The screenshot shows the SQLite Studio SQL Editor interface. The title bar reads 'SQLite Studio — SQL Editor' and 'Database: world.db'. Below the title bar is a toolbar with 'Execute (F9)', 'Database', 'Tools', and 'View' buttons. The main editor area is titled 'Lab 4 — Query Data with SELECT' and contains the following SQL query:

```
QUERY
SELECT * FROM City
ORDER BY Population DESC
LIMIT 10;
```

Below the query editor, the 'RESULTS' section displays a table with 5 columns: ID, Name, CountryCode, District, and Population. The table contains 10 rows of data, sorted by Population in descending order.

ID	Name	CountryCode	District	Population
1024	Mumbai	IND	Maharashtra	10500000
2331	Seoul	KOR	Seoul	9981619
206	Sao Paulo	BRA	Sao Paulo	9968485
1890	Shanghai	CHN	Shanghai	9696300
939	Jakarta	IDN	Jakarta Raya	9604900
2822	Karachi	PAK	Sindh	9269265

At the bottom of the results section, a status bar indicates: 'Query executed successfully · 10 rows returned · sorted by Population DESC'. The bottom right corner of the window shows the text 'SQL Fundamental for Beginners · TGS-2020505790'.

Figure 4 — Lab 4: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. In the world database's SQL editor, retrieve every row and column of City, then only the Name and Population columns.

```
SELECT * FROM City;
```

```
SELECT Name, Population FROM City;
```

Step 2. List each country code just once with SELECT DISTINCT.

```
SELECT DISTINCT CountryCode FROM City;
```

Step 3. Filter rows with WHERE — all cities in the district of Kabul.

```
SELECT * FROM City WHERE District = 'Kabul';
```

Step 4. Answer: what are the top 10 cities by population, largest first?

```
SELECT * FROM City
ORDER BY Population DESC
LIMIT 10;
```

Step 5. Answer: show 5 countries' name, continent and region where the continent is Europe, ordered by name.

```
SELECT Name, Continent, Region
FROM Country
WHERE Continent = 'Europe'
ORDER BY Name
LIMIT 5;
```

Test it

Each query runs without error; the top-10 query returns 10 rows led by the world's most populous cities, and the Europe query returns 5 alphabetically-ordered European countries.

Note: The same steps are in `courseware/labs/lab-04-*.md` in the course repository.

Lab 5 — Filter with SQL Operators

Objective: LO2 — analyse data with operators, patterns and sorting (A3, A5).

Goal: Sharpen your WHERE clauses with AND/OR/NOT, IN, BETWEEN, LIKE wildcards, NULL tests and aliases to slice the world data precisely.

What you'll build

A query toolkit covering every SQL operator, answering pattern- and range-based questions on the world database. (Tools: SQLite Studio, world database.)

SQLite Studio — SQL Editor

Database: world.db

Execute (F9)

Database

Tools

View

Lab 5 — Filter with SQL Operators

QUERY

```

SELECT Name, Continent, Population
FROM Country
WHERE Name LIKE '%island%'
ORDER BY Name;

```

RESULTS

Name	Continent	Population
Christmas Island	Oceania	2500
Cocos (Keeling) Islands	Oceania	600
Cook Islands	Oceania	20000
Iceland	Europe	279000
Marshall Islands	Oceania	64000
Solomon Islands	Oceania	444000

Query executed successfully · LIKE '%island%' matched 12 rows

SQL Fundamental for Beginners · TGS-2020505790

Figure 5 — Lab 5: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Combine conditions with AND, OR and NOT — cities in Afghanistan with population above 1,000,000.

```

SELECT * FROM City
WHERE CountryCode = 'AFG' AND Population > 1000000;

```

Step 2. Match a list of values with IN — cities in Afghanistan or the Netherlands.

```

SELECT * FROM City
WHERE CountryCode IN ('AFG', 'NLD');

```

Step 3. Select a range with BETWEEN — cities with population between 500,000 and 1,000,000.

```

SELECT Name, Population FROM City
WHERE Population BETWEEN 500000 AND 1000000;

```

Step 4. Find patterns with LIKE — countries whose name contains 'island' (the % wildcard matches any characters).

```

SELECT Name, Continent, Population
FROM Country

```

```
WHERE Name LIKE '%island%'
```

```
ORDER BY Name;
```

Step 5. Answer: countries' name, continent and population where the continent is Europe or Asia, ordered by name.

```
SELECT Name, Continent, Population
```

```
FROM Country
```

```
WHERE Continent IN ('Europe', 'Asia')
```

```
ORDER BY Name;
```

Step 6. Test for missing values with IS NULL — countries with no recorded independence year — and rename a column with an alias.

```
SELECT Name AS CountryName, IndepYear
```

```
FROM Country
```

```
WHERE IndepYear IS NULL;
```

Test it

The IN query returns only AFG and NLD cities; the LIKE query lists island nations (Iceland, Marshall Islands, ...); the IS NULL query returns countries with an empty IndepYear, shown under the alias CountryName.

Note: The same steps are in courseware/labs/lab-05-*.md in the course repository.

Lab 6 — Insert, Update and Delete Records

Objective: LO2 — process data by inserting, updating and deleting records (A3, A5).

Goal: Change data safely: INSERT new city rows, UPDATE them with a WHERE clause, DELETE a specific record, and load the sample database from the provided insert script.

What you'll build

A modified City table proving you can add, change and remove records — plus a fully-loaded sample database. (Tools: SQLite Studio, world & sample databases, insert script from the LMS.)

SQLite Studio — SQL Editor

Database: world.db

Execute (F9)

Database

Tools

View

Lab 6 — Insert, Update and Delete Records

QUERY

```

INSERT INTO City (Name, CountryCode)
VALUES ('Singapore', 'SGP');

SELECT * FROM City
WHERE Name = 'Singapore';

```

RESULTS

ID	Name	CountryCode	District	Population
4080	Singapore	SGP		0

1 row inserted · District and Population took their DEFAULT values

SQL Fundamental for Beginners · TGS-2020505790

Figure 6 — Lab 6: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Insert a new city into the City table, supplying just the columns you know — the rest take their defaults.

```

INSERT INTO City (Name, CountryCode)
VALUES ('Singapore', 'SGP');

```

Step 2. Verify the insert — find your new row.

```

SELECT * FROM City WHERE Name = 'Singapore';

```

Step 3. Update records with a WHERE clause — rename every city in the Kabol district.

```

UPDATE City
SET Name = 'Kabol'
WHERE District = 'Kabol';

```

Step 4. Delete one specific record by its primary key. (Never run DELETE without WHERE — it removes every row.)

```

DELETE FROM City WHERE ID = 3285;

```


Step 5. Load the sample database: download the insert-data script (world-inserts.sql) from the course materials on the LMS, open it in the sample database's SQL editor and execute it to populate City, Country and CountryLanguage.

Step 6. Confirm the load with row counts.

```
SELECT COUNT(*) FROM City;
```

```
SELECT COUNT(*) FROM Country;
```

```
SELECT COUNT(*) FROM CountryLanguage;
```

Test it

The Singapore row appears after the INSERT; the Kabul-district rows show the updated name; row ID 3285 is gone; and the three COUNT(*) queries return non-zero row counts in the sample database.

Note: The same steps are in courseware/labs/lab-06-*.md in the course repository.

Topic 03 — Data Transformation

Aggregate Data · Join Data · Group Data (TSC mapping: A4, A6, K5, K6)

Key concepts

- Aggregate functions — COUNT, AVG and SUM collapse many rows into one summary value.
- Joins — INNER, LEFT, RIGHT and FULL OUTER joins combine rows from two tables on a matching key.
- Set operations — UNION stacks the result-sets of two SELECTs with matching columns.
- Grouping — GROUP BY aggregates per group; HAVING filters groups the way WHERE filters rows.

Lab 7 — Aggregate Data with COUNT, AVG and SUM

Objective: LO3 — transform data with aggregate functions (A4, A6).

Goal: Summarise the world database with COUNT, AVG and SUM: count rows, average populations, total populations, and combine aggregates with DISTINCT and WHERE for real analytical questions.

What you'll build

A summary report of world statistics — city counts, average and total populations, and filtered aggregate answers. (Tools: SQLite Studio, world database.)

SQLite Studio — SQL Editor

Database: world.db

Execute (F9)

Database

Tools

View

Lab 7 — Aggregate Data with COUNT, AVG and SUM

QUERY

```

SELECT COUNT(CountryCode) AS Cities,
       AVG(Population) AS AvgPop,
       SUM(Population) AS TotalPop
FROM City;

```

RESULTS

Cities	AvgPop	TotalPop
4079	350468.2	1429559884

Query executed successfully · 1 row returned · aggregates over 4,079 cities

SQL Fundamental for Beginners · TGS-2020505790

Figure 7 — Lab 7: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Count the rows in City.

```
SELECT COUNT(CountryCode) FROM City;
```

Step 2. Average a numeric column — the mean population of a country.

```
SELECT AVG(Population) FROM Country;
```

Step 3. Total a numeric column — the world's population.

```
SELECT SUM(Population) FROM Country;
```

Step 4. Answer: how many distinct heads of state are there?

```
SELECT COUNT(DISTINCT HeadOfState) FROM Country;
```

Step 5. Answer: how many countries in Europe have a population above 100,000,000?

```

SELECT COUNT(*) FROM Country
WHERE Continent = 'Europe'
AND Population > 100000000;

```

Test it

Each aggregate returns a single-value result-set; the distinct head-of-state count is smaller than the country count, and the Europe query returns a small whole number.

Note: The same steps are in courseware/labs/lab-07-*.md in the course repository.

Lab 8 — Join Data from Multiple Tables

Objective: LO3 — transform data from multiple sources with joins and unions (A4, A6).

Goal: Combine tables: INNER JOIN City to Country on the country code, see how LEFT JOIN keeps unmatched rows, and emulate a FULL OUTER JOIN in SQLite using LEFT JOIN + UNION on a pair of practice tables.

What you'll build

Joined result-sets linking cities to their countries, plus a full-outer-join of two practice tables showing matched and unmatched rows. (Tools: SQLite Studio, world & sample databases.)

The screenshot shows the SQLite Studio interface. The title bar reads 'SQLite Studio — SQL Editor' and 'Database: world.db'. Below the title bar is a menu bar with 'Execute (F9)', 'Database', 'Tools', and 'View'. The main area is titled 'Lab 8 — Join Data from Multiple Tables' and contains a SQL query. Below the query is a 'RESULTS' section showing a table with two columns: 'CityName' and 'CountryName'. The table contains six rows of data. At the bottom, a status bar indicates 'Query executed successfully · 20 rows returned · INNER JOIN on CountryCode = Code' and 'SQL Fundamental for Beginners · TGS-2020505790'.

```
QUERY
SELECT City.Name AS CityName,
       Country.Name AS CountryName
FROM City
INNER JOIN Country
  ON City.CountryCode = Country.Code
LIMIT 20;
```

CityName	CountryName
Kabul	Afghanistan
Qandahar	Afghanistan
Amsterdam	Netherlands
Rotterdam	Netherlands
Oranjestad	Aruba
Tirana	Albania

Query executed successfully · 20 rows returned · INNER JOIN on CountryCode = Code

SQL Fundamental for Beginners · TGS-2020505790

Figure 8 — Lab 8: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Inner join City and Country on city.CountryCode = country.Code, retrieving the country code and district.

```
SELECT Country.Code, City.District
```

```
FROM City
INNER JOIN Country
    ON City.CountryCode = Country.Code;
```

Step 2. Extend the join to name both sides — each city with its country's name.

```
SELECT City.Name AS CityName,
       Country.Name AS CountryName
FROM City
INNER JOIN Country
    ON City.CountryCode = Country.Code
LIMIT 20;
```

Step 3. LEFT JOIN keeps every row of the left table even without a match — every country and (where present) its capital city.

```
SELECT Country.Name, City.Name AS Capital
FROM Country
LEFT JOIN City
    ON Country.Capital = City.ID
ORDER BY Country.Name;
```

Step 4. Create two small practice tables to demonstrate a full outer join.

```
CREATE TABLE left_t ( id INTEGER, description TEXT );
CREATE TABLE right_t ( id INTEGER, description TEXT );
INSERT INTO left_t VALUES (1,'left 01'),(2,'left 02'),(3,'left 03'),
(4,'left 04');
INSERT INTO right_t VALUES (3,'right 03'),(4,'right 04'),(5,'right 05'),
(6,'right 06');
```

Step 5. SQLite has no FULL OUTER JOIN keyword — emulate it with a LEFT JOIN in each direction combined by UNION.

```
SELECT l.id, l.description, r.description
FROM left_t l LEFT JOIN right_t r ON l.id = r.id
UNION
SELECT r.id, l.description, r.description
```

```
FROM right_t r LEFT JOIN left_t l ON l.id = r.id

ORDER BY id;
```

Test it

The inner join returns only matched city–country rows; the LEFT JOIN lists every country (NULL capital where unmatched); and the full-join emulation returns ids 1–6 with NULLs on the sides that have no match.

Note: The same steps are in courseware/labs/lab-08-*.md in the course repository.

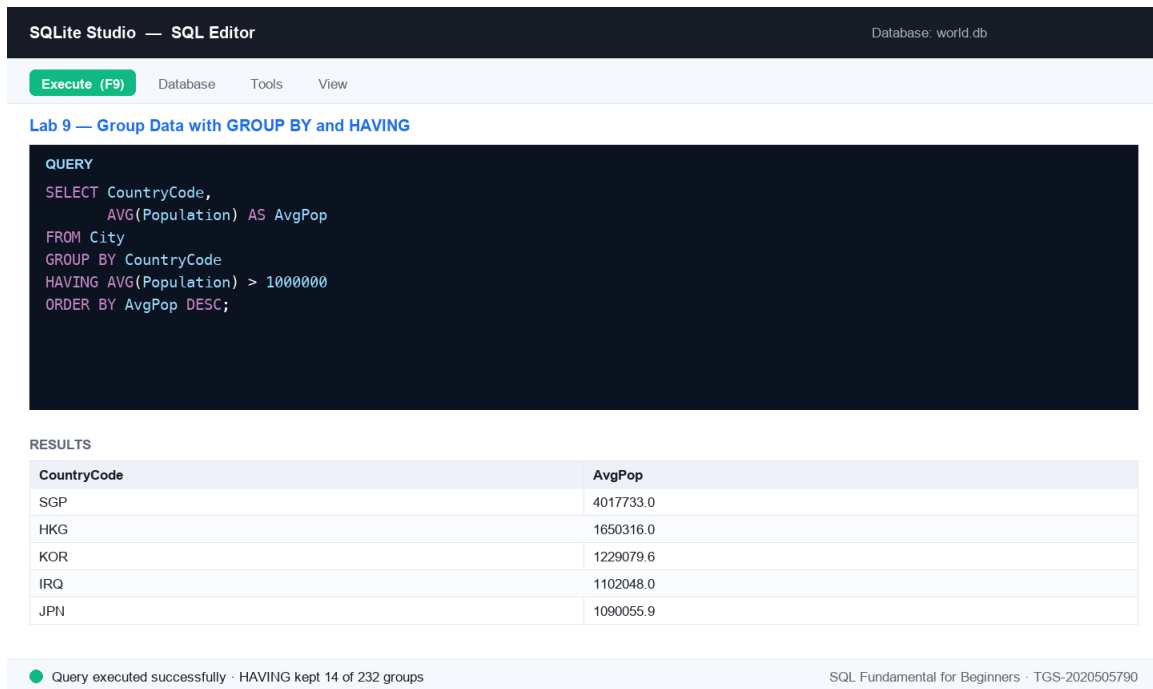
Lab 9 — Group Data with GROUP BY and HAVING

Objective: LO3 — transform data by grouping and filtering groups (A4, A6).

Goal: Aggregate per group: average city population by country code with GROUP BY, then keep only the populous groups with HAVING — and see why HAVING exists where WHERE cannot go.

What you'll build

Grouped summaries of the world data — per-country averages, filtered to the groups that matter. (Tools: SQLite Studio, world database.)



The screenshot shows the SQLite Studio SQL Editor interface. The title bar indicates 'SQLite Studio — SQL Editor' and 'Database: world.db'. The menu bar includes 'Execute (F9)', 'Database', 'Tools', and 'View'. The main editor area displays a SQL query for Lab 9: Group Data with GROUP BY and HAVING. The query selects CountryCode and the average population (AvgPop) from the City table, grouped by CountryCode, with a HAVING clause filtering for groups with an average population greater than 1,000,000, ordered by average population in descending order. Below the query, the 'RESULTS' section shows a table with two columns: CountryCode and AvgPop. The results list five countries: SGP, HKG, KOR, IRQ, and JPN, with their respective average populations. A status bar at the bottom indicates 'Query executed successfully · HAVING kept 14 of 232 groups' and 'SQL Fundamental for Beginners · TGS-2020505790'.

```
QUERY
SELECT CountryCode,
       AVG(Population) AS AvgPop
FROM City
GROUP BY CountryCode
HAVING AVG(Population) > 1000000
ORDER BY AvgPop DESC;
```

CountryCode	AvgPop
SGP	4017733.0
HKG	1650316.0
KOR	1229079.6
IRQ	1102048.0
JPN	1090055.9

Query executed successfully · HAVING kept 14 of 232 groups

SQL Fundamental for Beginners · TGS-2020505790

Figure 9 — Lab 9: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Retrieve the average city population grouped by country code.

```
SELECT CountryCode, AVG(Population)
FROM City
GROUP BY CountryCode;
```

Step 2. Add a readable alias and order the summary, largest first.

```
SELECT CountryCode,
        AVG(Population) AS AvgPop
FROM City
GROUP BY CountryCode
ORDER BY AvgPop DESC;
```

Step 3. Keep only groups whose average exceeds 1,000,000 — WHERE cannot filter aggregates, HAVING can.

```
SELECT CountryCode, AVG(Population) AS AvgPop
FROM City
GROUP BY CountryCode
HAVING AVG(Population) > 1000000
ORDER BY AvgPop DESC;
```

Step 4. Combine everything: count cities per country, but only for countries with more than 50 recorded cities.

```
SELECT CountryCode, COUNT(*) AS Cities
FROM City
GROUP BY CountryCode
HAVING COUNT(*) > 50
ORDER BY Cities DESC;
```

Test it

The GROUP BY query returns one row per country code; the HAVING queries return only the qualifying groups (every AvgPop > 1,000,000; every Cities > 50), sorted descending.

Note: The same steps are in `courseware/labs/lab-09-*.md` in the course repository.

Topic 04 — Introduction to Data Warehouse

Overview of Data Warehouse · Relational Databases · Stored Procedures (TSC mapping: A7, A8, A9, K3, K7)

Key concepts

- Data warehouse — A consolidated, historical, read-mostly database kept separate from operational systems for analysis.
- OLAP vs OLTP — OLAP serves complex analytical queries; OLTP serves fast day-to-day transactions.
- RDBMS & E-R model — The relational model plus entity-relationship design maps business entities to tables and keys.
- Stored procedures — Prepared SQL saved on the server (SQL Server) for reuse — callable with parameters via EXEC.

Lab 10 — Map an E-R Model to Database Tables (SMRT Case Study)

Objective: LO4 — apply data mapping from an E-R model to a data warehouse schema (A7, A8, A9).

Goal: Take the SMRT public-transport E-R model (Routes, Timetables, DisruptedRoutes) and map it to physical tables: create each entity with its primary key, wire the foreign-key relationships, and verify the model with joined queries.

What you'll build

A three-table transport schema in SQLite that faithfully implements the SMRT E-R model with PK/FK mappings. (Tools: SQLite Studio, sample database.)

SQLite Studio — SQL Editor

Database: sample.db

Execute (F9)

Database

Tools

View

Lab 10 — Map an E-R Model to Database Tables (SMRT Case Study)

QUERY

```

SELECT r.route_name, t.station_code,
       d.disrupt_name, d.duration
FROM DisruptedRoutes d
INNER JOIN Timetables t ON d.timetable_id = t.timetable_id
INNER JOIN Routes r     ON t.route_id     = r.route_id;

```

RESULTS

route_name	station_code	disrupt_name	duration
North South Line	NS01	Signal fault	45 min

Query executed successfully · 3-table join across the SMRT E-R model

SQL Fundamental for Beginners · TGS-2020505790

Figure 10 — Lab 10: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Review the E-R model: Routes (route_id PK) 1—many Timetables (timetable_id PK, route_id FK) 1—many DisruptedRoutes (disrupt_no PK, timetable_id FK). Identify each entity's attributes before writing any SQL.

Step 2. Create the Routes parent table.

```

CREATE TABLE Routes (
    route_id CHARACTER(4) NOT NULL PRIMARY KEY,
    route_type CHARACTER(15) NOT NULL,
    route_code VARCHAR(3) NOT NULL,
    route_name VARCHAR(125) NOT NULL,
    route_direction CHARACTER(10) NOT NULL,
    remarks VARCHAR(255)
);

```

Step 3. Create the Timetables child table — its route_id column references Routes.

```

CREATE TABLE Timetables (
    timetable_id INTEGER NOT NULL PRIMARY KEY,

```



```

route_id CHARACTER(4) NOT NULL,
station_code CHARACTER(4) NOT NULL,
frequency VARCHAR(20) DEFAULT '',
days_operation VARCHAR(30) NOT NULL,
stop_no NUMERIC(3) NOT NULL,
first_trip CHAR(10) NOT NULL,
last_trip CHAR(10) NOT NULL,
FOREIGN KEY (route_id) REFERENCES Routes(route_id)
);

```

Step 4. Create the DisruptedRoutes table — its timetable_id references Timetables.

```

CREATE TABLE DisruptedRoutes (
    disrupt_no INTEGER NOT NULL PRIMARY KEY,
    disrupt_date CHAR(10) NOT NULL,
    disrupt_type VARCHAR(20) NOT NULL,
    disrupt_name VARCHAR(50) NOT NULL,
    disrupt_details VARCHAR(300) NOT NULL,
    start_datetime CHAR(12) NOT NULL,
    duration CHAR(20) NOT NULL,
    timetable_id INTEGER NOT NULL,
    alternatives VARCHAR(3000) NOT NULL,
    FOREIGN KEY (timetable_id) REFERENCES Timetables(timetable_id)
);

```

Step 5. Load one row into each table to exercise the relationship chain.

```

INSERT INTO Routes VALUES ('R001','Train','NSL','North South
Line','North','');

```

```

INSERT INTO Timetables VALUES (1,'R001','NS01','5 min','Mon-
Sun',1,'05:30','23:30');

```

```

INSERT INTO DisruptedRoutes VALUES (1,'2024-03-14','system','Signal
fault','Signalling fault at NS01','202403140800','45 min',1,'Bridging bus
1A');

```

Step 6. Verify the mapping end-to-end: join all three tables from disruption back to route.

```
SELECT r.route_name, t.station_code,  
       d.disrupt_name, d.duration  
FROM DisruptedRoutes d  
INNER JOIN Timetables t ON d.timetable_id = t.timetable_id  
INNER JOIN Routes r ON t.route_id = r.route_id;
```

Test it

The three tables exist with their PK/FK constraints visible in the DDL, and the three-way join returns one row linking 'North South Line' → 'NS01' → 'Signal fault'.

Note: The same steps are in courseware/labs/lab-10-*.md in the course repository.

Lab 11 — Author Stored Procedures for SQL Server

Objective: LO4 — operate data-warehouse platform capabilities with reusable stored procedures (A7, A8, A9).

Goal: Write stored procedures in SQL Server syntax — a plain procedure and a parameterised one — and understand where they run (SQL Server / MySQL, not SQLite) and why warehouses use them for repeatable batch work.

What you'll build

Two stored-procedure scripts (plain and parameterised) ready to run on SQL Server, plus the EXEC calls that invoke them. (Tools: SQL Editor, SQL Server syntax (sqliteonline.com's MS SQL engine, or any SQL Server).)

SQLite Studio — SQL Editor

Database: MS SQL (sqliteonline.com)

Execute (F9)

Database

Tools

View

Lab 11 — Author Stored Procedures for SQL Server

QUERY

```

CREATE PROCEDURE SelectCustomersByCity
    @City nvarchar(30)
AS
SELECT * FROM Customers
WHERE City = @City
GO;

EXEC SelectCustomersByCity @City = 'London';

```

RESULTS

CustomerID	CustomerName	ContactName	City	Country
4	Around the Horn	Thomas Hardy	London	UK
11	B's Beverages	Victoria Ashworth	London	UK
16	Consolidated Holdings	Elizabeth Brown	London	UK

● Procedure created and executed - 6 rows returned for @City = 'London'

SQL Fundamental for Beginners · TGS-2020505790

Figure 11 — Lab 11: the SQL editor after running this lab's key statement.

Step-by-step

Step 1. Stored procedures are not available in SQLite — author these scripts against SQL Server. (Free option: sqliteonline.com and switch the engine to MS SQL, which provides a Customers demo table.)

Step 2. Write a stored procedure that returns all customers, then save it.

```
CREATE PROCEDURE SelectAllCustomers
```

```
AS
```

```
SELECT * FROM Customers
```

```
GO;
```

Step 3. Execute the stored procedure by name.

```
EXEC SelectAllCustomers;
```

Step 4. Write a parameterised version that filters customers by city through a @City parameter.

```
CREATE PROCEDURE SelectCustomersByCity
```

```
    @City nvarchar(30)
```

```
AS
```

```
SELECT * FROM Customers  
  
WHERE City = @City  
  
GO;
```

Step 5. Execute it, passing the parameter value.

```
EXEC SelectCustomersByCity @City = 'London';
```

Test it

Both procedures create without error; EXEC SelectAllCustomers returns the full customer list, and EXEC SelectCustomersByCity @City = 'London' returns only the London rows.

Note: The same steps are in courseware/labs/lab-11-*.md in the course repository.

SQL Quick Reference

The statements taught in this course, in one place. All run in SQLite unless marked otherwise.

Define structure (DDL)

```
CREATE DATABASE dbname;           -- create a database (file in  
SQLite)  
  
CREATE TABLE t (col TYPE constraint, ...); -- create a table  
  
ALTER TABLE t ADD col TYPE;      -- add a column  
  
CREATE INDEX idx ON t (col);      -- speed up searches  
  
DROP TABLE t; DROP DATABASE dbname; -- delete (careful!)
```

Constraints

```
NOT NULL | UNIQUE | DEFAULT 'value' | CHECK (cond)  
  
PRIMARY KEY (col)  
  
FOREIGN KEY (col) REFERENCES parent(pk)
```

Query (DQL)

```
SELECT col1, col2 FROM t;          -- chosen columns ( * = all )  
  
SELECT DISTINCT col FROM t;        -- unique values  
  
SELECT * FROM t WHERE cond;        -- filter rows
```

```

cond: = <> > < >= <= AND OR NOT
      IN (v1,v2) BETWEEN a AND b
      LIKE 'a%' IS NULL EXISTS (subquery)

SELECT col AS alias FROM t;           -- rename in the result
ORDER BY col ASC|DESC LIMIT n        -- sort and cap

```

Change data (DML)

```

INSERT INTO t (c1,c2) VALUES (v1,v2);

UPDATE t SET c1 = v1 WHERE cond;      -- ALWAYS use WHERE
DELETE FROM t WHERE cond;             -- ALWAYS use WHERE

```

Transform (aggregate · join · group)

```

SELECT COUNT(c), AVG(c), SUM(c) FROM t WHERE cond;

SELECT ... FROM t1 INNER JOIN t2 ON t1.k = t2.k;

SELECT ... FROM t1 LEFT JOIN t2 ON t1.k = t2.k;

SELECT ... UNION SELECT ...;          -- same columns both sides

GROUP BY col HAVING AGG(col) cond     -- filter groups, not rows

```

Stored procedures (SQL Server — not SQLite)

```

CREATE PROCEDURE name @param type AS sql_statement GO;

EXEC name @param = value;

```

After the Course

- Redo every lab from memory in SQLite Studio — fluency comes from typing the SQL yourself.
- Point SQLite Studio at your own data: import a CSV into a table and analyse it with the Topic 2–3 queries.
- The same SQL works in MySQL, PostgreSQL and SQL Server — try the world database on a server engine.
- Continue with the recommended WSQ courses at <https://www.tertiarycourses.com.sg>.
- Complete the TRAQOM feedback survey and download your e-certificate from <https://lms-tms.tertiaryinfotech.com> after being assessed Competent.

Glossary

- **Database** — An organised collection of data — in SQLite, a single .db file holding tables.
- **Table** — A collection of rows sharing the same columns (fields); the unit SQL queries operate on.
- **Row / Record** — One instance of a table's row type — the smallest unit inserted or deleted.
- **Column / Field** — A named, typed slot that every row of a table has.
- **Result-set** — The table of rows returned by a SELECT query.
- **Constraint** — A rule on a column (NOT NULL, UNIQUE, CHECK, DEFAULT, keys) enforced by the engine.
- **Primary key** — The column(s) that uniquely identify each row — UNIQUE + NOT NULL, one per table.
- **Foreign key** — A column referencing another table's primary key, linking child to parent.
- **Index** — A hidden structure that speeds up searches on the indexed column(s).
- **Join** — Combining rows of two tables on matching key values (INNER/LEFT/RIGHT/FULL).
- **Aggregate function** — COUNT, AVG, SUM, MIN, MAX — collapse many rows into one summary value.
- **GROUP BY / HAVING** — Aggregate per group of equal values / filter those groups.
- **Data modeling** — Designing entities, attributes and key relationships to mirror a business process.
- **E-R model** — Entity-Relationship model — the high-level diagram mapped into tables and keys.
- **Data warehouse** — A separate, consolidated, historical, read-mostly database for analysis and decisions.
- **OLAP / OLTP** — Analytical processing over the warehouse / transactional processing in operational systems.
- **RDBMS** — Relational Database Management System — the engine class SQL was built for.
- **Stored procedure** — Prepared SQL saved on the server (SQL Server/MySQL), executed by name with EXEC.
- **SQL injection** — Attacking a system by smuggling SQL through user input — prevented with parameterised queries.